

Full Length Article

PGMNO: A physics-Guided mamba neural operator framework for partial differential equations

Yanan Guo ^{a,b,c}, Junqiang Song ^{a,b,*}, Xiaoqun Cao ^{a,*}, Chuanfeng Zhao ^c,
Fukang Yin ^a, Hongze Leng ^a

^a College of Meteorology and Oceanography, National University of Defense Technology, No.109 Deya Road, Changsha, 410073, Hunan, China

^b College of Computer Science, National University of Defense Technology, No.109 Deya Road, Changsha, 410073, Hunan, China

^c Department of Atmospheric and Oceanic Sciences, School of Physics, Peking University, No.5 Yiheyuan Road, Beijing, 100871, Beijing, China

ARTICLE INFO

PACS:

47.11.-J

02.60.Cb

02.70.-C

2000 MSC:

35Q30

65M70

68T07

Keywords:

Physics-Guided neural operator

State space model

Multistep method

PDE Learning

Scientific machine learning

Operator learning

ABSTRACT

Accurately modeling the long-term evolution of complex physical systems governed by partial differential equations (PDEs) remains a central challenge in operator learning. In this work, we introduce the Physics-Guided Mamba Neural Operator (PGMNO), a framework specifically designed to capture long-range spatiotemporal dependencies. PGMNO overcomes the limitations of conventional single-step and Transformer-based operators by unifying linear multistep numerical methods with structured state space models (SSMs). To ensure temporal stability, the framework employs multistep temporal modeling in the forward pass, complemented by an implicit backward differentiation formula (BDF)-based scheme during training. Simultaneously, PGMNO leverages the efficient kernel integration properties of SSMs to achieve scalable and stable operator learning. We validate PGMNO on diverse PDE benchmarks. Experimental results demonstrate that PGMNO consistently outperforms state-of-the-art models in prediction accuracy, computational efficiency, and long-term stability. Additionally, the framework exhibits strong resolution-invariant extrapolation and generalization across varying spatial discretizations. By prioritizing clear methodological articulation, this work underscores the potential of unifying numerical integration with state space modeling to build robust surrogate solvers for PDE-governed dynamical systems.

1. Introduction

Partial differential equations (PDEs) provide the mathematical foundation for modeling complex physical and engineering systems. These include fluid dynamics, electromagnetism, reaction-diffusion phenomena, and biomechanics (Evans, 2022; Fefferman et al., 2018; Olver et al., 2014). In practice, such PDEs are typically solved by discretizing space and time using methods such as finite differences, finite elements, or spectral methods (Ames, 2014; Johnson, 2009; Thomas, 2013). While these classical solvers offer rigorous error control and high accuracy, they become prohibitively expensive as the problem dimensionality or grid resolution increases. This “curse of dimensionality” often renders high-fidelity simulations infeasible in many-query contexts, such as real-time control or large-scale parameter sweeps. Furthermore, many critical systems exhibit strong nonlinearity or stiffness. For example, the Navier–Stokes equations often necessitate extremely small time steps or

sophisticated stabilization techniques, which further amplify the total computational cost.

To address these challenges, three broad strategies have recently emerged to accelerate PDE simulations. The first approach focuses on purely data-driven surrogate modeling (Bar-Sinai et al., 2019; Dal Santo et al., 2020). In this regime, neural networks are trained to map problem inputs—such as initial conditions or material parameters—directly to solution fields. Convolutional neural networks (CNNs) have proven effective for data on structured grids (Zha et al., 2022), while recurrent neural networks (RNNs) are utilized to capture temporal dynamics (Huang et al., 2025). Additionally, Transformer-based architectures leverage self-attention to model long-range dependencies (Li et al., 2023b). However, purely data-driven surrogates typically require massive training datasets and may suffer from poor generalization. They can also violate conservation laws or physical invariants when extrapolating beyond the train-

* Corresponding author.

E-mail addresses: junqiang@nudt.edu.cn (J. Song), caoxiaoqun@nudt.edu.cn (X. Cao).

<https://doi.org/10.1016/j.neunet.2026.108845>

Received 29 July 2025; Received in revised form 29 January 2026; Accepted 10 March 2026

Available online 21 March 2026

0893-6080/© 2026 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

ing distribution. Moreover, standard Transformer architectures incur a computational cost that scales quadratically with sequence length (Su et al., 2025), limiting their applicability to long-duration simulations.

The second approach is physics-informed learning, which embeds the known PDE structure directly into the model or its loss function. For instance, physics-informed neural networks (PINNs) incorporate the PDE residual as a penalty term in the training objective (Guo et al., 2023; Luo et al., 2025; Zhongkai et al., 2024). This strategy enforces consistency with the governing equations and reduces the reliance on labeled data. In practice, PINNs impose the PDE residual through automatic differentiation or finite-difference discretizations. While successful in data-scarce regimes and inverse problems, PINNs often involve highly nonconvex optimization landscapes. This can lead to slow convergence for stiff or turbulent systems, where existing formulations may stagnate at unphysical local minima. Furthermore, these architectures typically learn only a single solution instance and must be retrained for each new initial condition or parameter setting. By contrast, universal neural operator models aim to generalize across diverse input conditions without the need for case-by-case retraining.

The third strategy is neural operator learning (Kovachki et al., 2023; Li et al., 2023a; Wang & Wang, 2024), which models mappings between infinite-dimensional function spaces. A neural operator approximates the true solution operator $\mathcal{O}$ that transforms input functions into the corresponding solution fields. By targeting the solution operator directly, these models can generalize across various input conditions and spatial resolutions. Representative architectures include DeepONet (Kumar et al., 2025), Fourier Neural Operators (FNO) (Kovachki et al., 2021), and various graph-based models. DeepONet employs a dual-network design: a branch network encodes the input function, while a trunk network encodes the query coordinates. Their inner product yields the solution value at the desired point. Under suitable conditions, DeepONet serves as a universal approximator for continuous nonlinear operators. Extensions such as MIONet further utilize multiple branch networks to handle coupled multi-physics systems.

The Fourier Neural Operator (FNO) takes a contrasting approach by learning convolution kernels in the spectral domain. Specifically, FNO applies a fast Fourier transform (FFT) to the input field and performs a linear transformation on the low-frequency modes. The representation is then returned to physical space via an inverse FFT. This global, spectral formulation provides a wide receptive field and enables efficient parallel computation. In benchmark tests, FNO has accurately modeled turbulent flows with zero-shot super-resolution. However, its reliance on the FFT restricts its use to uniform Cartesian grids and limits its applicability to irregular domains. Recent work has addressed these issues through domain deformation (Geo-FNO) or by formulating the operator on arbitrary point clouds. Graph-based models and attention-based transformers have also been developed to handle complex geometries (Li et al., 2020). For example, Graph Kernel Networks (GKNs) use message passing on unstructured meshes to learn PDE mappings (Anandkumar et al., 2019). Similarly, hierarchical transformer architectures, such as HT-Net, capture multi-scale features by combining self-attention with cross-attention.

Specialized transformer variants have further improved efficiency. The Transolver architecture, for example, uses a physics-aware attention mechanism to partition high-dimensional meshes into latent slices (Wu et al., 2024). This partitioning ensures that inference costs grow only linearly with the number of mesh points. While such approaches remain stable across varying resolutions, significant limitations persist in current neural operator methods. Purely spectral approaches often struggle with non-Cartesian domains, and introducing physics-informed constraints can increase training complexity. Moreover, long-time integration remains a central challenge, as small prediction errors can accumulate and degrade the solution over time. Since the choice of se-

quence modeling architecture directly dictates the stability and forecast horizon of time-dependent systems, researchers have increasingly explored more expressive structures to capture complex nonlinear trends. For instance, in macroeconomic forecasting, sequence models incorporating attention mechanisms have been shown to significantly enhance the modeling of complex temporal dependencies (Adamopoulos et al., 2025). Beyond traditional attention-based architectures, the emergence of structured state space models (SSMs) has provided a potent alternative for continuous-time modeling due to their linear-time complexity and superior memory capacity. A notable advancement is DyG-Mamba (Li et al., 2024), which implements continuous-time selective SSMs with input-dependent gating to resolve long-range dependencies in dynamic graphs. By successfully capturing complex evolutionary patterns through a selective state-update mechanism, DyG-Mamba substantiates the inherent robustness of SSMs in modeling non-stationary dynamics. This framework inspires related research to integrate these adaptive mechanisms into physics-guided operator learning, thereby facilitating stable long-horizon prediction of partial differential equations (PDEs) in complex physical regimes.

Despite the rapid evolution of these operators, a fundamental disconnect persists between the discrete-time nature of numerical simulations and the continuous-space representation of neural architectures. Current paradigms often treat temporal evolution either as a simple recursive mapping or through soft-constrained residuals in the loss function. This lack of explicit temporal structure frequently leads to error accumulation and physical inconsistency in long-duration forecasts. As demonstrated in recent fluid dynamics research, such as the analysis of activation energy impacts on Sisko fluid flows (Rehman et al., 2025), the explicit articulation of coupling mechanisms between physical governing laws and computational frameworks is essential for establishing scientific novelty and ensuring model reliability. In that context, highlighting how specific physical parameters interact with the flow field enhances the perception of a model's scientific contribution.

Building upon this perspective, we argue that the integration of physics should transcend simple loss regularization. While Physics-Informed Neural Operators (PINOs) utilize PDE residuals to penalize unphysical solutions, they do not inherently guarantee that the model's internal state evolution remains within a stable numerical manifold. Our work introduces the Physics-Guided Mamba Neural Operator (PGMNO), which achieves a deeper level of integration by embedding linear multi-step integration schemes directly into the operator's architecture. From a theoretical standpoint, this structural coupling functions as a physics-consistent inductive bias. By reformulating the latent evolution of the Mamba block to satisfy the algebraic structure of implicit numerical schemes, the model is constrained to follow trajectories that align with classical stability criteria. This approach ensures that the resulting operator is not merely a black-box regression tool but a structured approximation of the underlying physical propagator. Consequently, PGMNO provides a more rigorous foundation for capturing unsteady dynamics and long-term evolution than purely data-driven or loss-regularized alternatives.

In summary, while neural operators offer powerful tools for data-driven PDE modeling, key challenges remain in handling multiple input fields and ensuring long-term robustness. Addressing these gaps requires a rigorous methodological articulation that bridges the gap between classical Computational Fluid Dynamics (CFD) and modern Machine Learning (ML) (Hakim, 2024). Our work introduces the Physics-Guided Mamba Neural Operator to address these limitations. To ensure temporal stability, we incorporate an implicit backward differentiation formula (BDF)-based integration scheme into the training process. Simultaneously, we leverage the efficient kernel integration properties of structured state space models (SSMs) to achieve scalable and stable operator learning. This unified framework provides a coherent approach to building robust surrogate solvers for PDE-governed dynamical systems.

2. Related work

2.1. Neural operators

Neural operators are designed to learn mappings between infinite-dimensional function spaces, enabling direct approximation of PDE solution operators that map input functions (such as initial or boundary data) to output solution fields. Unlike traditional neural networks that map fixed-size input vectors to fixed-size outputs, neural operators can naturally generalize across varying input functions and spatial resolutions. Early work in this area includes the DeepONet architecture, which uses a two-network design (branch and trunk networks) to encode the input function and the query coordinates, respectively. Under suitable conditions, DeepONet is a universal approximator for continuous nonlinear operators. For problems with multiple input fields (such as coupled multi-physics PDEs), extensions like MIONet use separate branch networks for each input function, enabling the learning of operators with several inputs. The Fourier neural operator (FNO) operates in the spectral domain by parameterizing convolutional kernels via learnable Fourier multipliers. This yields a highly expressive model with a global receptive field and efficient (quasi-linear) evaluation via the fast Fourier transform. FNO has been shown to accurately model complex phenomena such as turbulent fluid flows, achieving impressive accuracy and even “zero-shot” super-resolution. However, FNO requires data on uniform Cartesian grids due to its reliance on the FFT. Several extensions have been proposed to relax this requirement, such as learned coordinate transforms (Geo-FNO) or spectral methods defined on arbitrary point clouds, to handle irregular geometries and unstructured data. In addition to spectral methods, attention-based neural operators have been developed. For instance, the Galerkin Transformer uses self-attention as a learnable integral kernel, allowing the operator to adapt to the input data. Hierarchical transformer networks (e.g. HT-Net) combine multi-scale encodings with cross-attention to capture features at multiple scales. The Transolver model employs a physics-aware attention mechanism to learn latent “slices” of the computational mesh, yielding a resolution-agnostic operator whose inference cost scales linearly with the number of mesh points. In benchmarks, Transolver remains accurate and efficient as the mesh is refined, in contrast to standard Transformers which become infeasible due to their quadratic complexity. Neural operator approaches continue to evolve, including hybrid spectral-attention models and physics-informed operator methods. For example, attention-enhanced Fourier neural operators (such as AFNO) add self-attention layers to FNO blocks to capture local features, and wavelet neural operators (WNOs) blend wavelet transforms with convolution to capture multiscale information. Physics-informed operator methods incorporate PDE residuals or conservation laws directly into the training objective. Nonetheless, core challenges remain: extending operator learning to multiple input/output fields, handling complex and irregular domains, and maintaining stability and accuracy over long-time integration.

2.2. Structured state-space models (SSMs)

Structured state-space models have recently emerged as powerful sequence models that rival Transformers on very long contexts, with much lower computational cost. The continuous-time SSM is defined by linear dynamics:

$$\dot{\mathbf{h}}(t) = \mathbf{A} \mathbf{h}(t) + \mathbf{B} \mathbf{x}(t), \quad (1)$$

$$\mathbf{y}(t) = \mathbf{C} \mathbf{h}(t) + \mathbf{D} \mathbf{x}(t), \quad (2)$$

where $\mathbf{x}(t)$ is the input signal, $\mathbf{h}(t)$ is the hidden state, and $\mathbf{y}(t)$ is the output. Here $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, and $\mathbf{D}$ are learned matrices.

Discretizing with a time step Δt under a zero-order hold (assuming constant input over each interval) yields the recurrence

$$\mathbf{h}_k = \bar{\mathbf{A}} \mathbf{h}_{k-1} + \bar{\mathbf{B}} \mathbf{x}_k, \quad (3)$$

$$\mathbf{y}_k = \mathbf{C} \mathbf{h}_k + \mathbf{D} \mathbf{x}_k, \quad (4)$$

where

$$\bar{\mathbf{A}} = \exp(\Delta t \mathbf{A}), \quad \bar{\mathbf{B}} = (\exp(\Delta t \mathbf{A}) - \mathbf{I}) \mathbf{A}^{-1} \mathbf{B}. \quad (5)$$

It has been observed that this discrete SSM is mathematically equivalent to a convolutional filter: the output sequence $\mathbf{y}$ is a dense linear convolution of the input $\mathbf{x}$ with a kernel $\mathbf{K}$, so that

$$\mathbf{y} = \mathbf{K} * \mathbf{x}. \quad (6)$$

In other words, a single SSM layer implements a global convolution on the entire input sequence, with kernel elements determined by powers of $\bar{\mathbf{A}}$.

Early SSM-based sequence models demonstrated that, with appropriate structure in the matrix $\mathbf{A}$, one can capture very long-range dependencies. However, a naive SSM implementation is computationally expensive when the hidden dimension N is large. The S4 model addressed this by introducing a structured parameterization of $\mathbf{A}$ (inspired by HiPPO memory matrices) and a low-rank correction that allows $\bar{\mathbf{A}}$ to be diagonalized stably. S4 effectively converts the SSM convolution into a fast computation via a Cauchy-like parameterization of the kernel. This enables very long convolution kernels (on the order of 10^5) to be applied in parallel, yielding runtime complexity that is linear (or $O(L \log L)$) in the sequence length. S4 achieved state-of-the-art results on many long-sequence benchmarks, successfully solving tasks of length up to 16K that were previously infeasible.

Subsequent advances have further improved SSM scalability. For example, S4D extends the diagonalization technique to higher-dimensional inputs (such as images), and other variants impose low-rank or Kronecker structure on the state matrices to reduce parameters. To capture input-dependent dynamics, the Selective State-Space model (S6) makes the state matrices $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ at each time step a learned function of the input embedding. This yields a time-varying convolution kernel where each output $\mathbf{y}_k$ effectively sees a different kernel slice. Although S6 loses the pure convolution form, efficient scan algorithms allow parallel computation, preserving an $O(L)$ runtime.

More recently, the Mamba model further incorporates selective SSMs into an end-to-end architecture without any self-attention. Mamba’s key idea is to let the SSM parameters themselves be input-dependent, effectively allowing the model to “selectively” propagate or forget information based on the current token. This results in a purely recurrent SSM network that remains linear-time in sequence length while achieving excellent performance on long sequences. For instance, Mamba models have matched or exceeded the performance of Transformers much larger in size on language modeling tasks, while delivering significantly faster throughput.

In scientific machine learning, SSMs have started to influence PDE modeling. They offer a natural way to fuse continuous-time dynamics with deep learning: an SSM layer can be viewed as a time-discretization of an ODE/PDE or as an integral operator over time. Some recent models draw analogies between graph neural networks and diffusion equations, and others have included SSM-based convolutional layers in spatiotemporal networks. More recently, “trans-SSM” architectures have emerged that use latent SSM kernels as integral operators acting on function inputs, effectively decoupling spatial resolution from computation. These developments suggest that structured SSMs could serve as scalable surrogate models for high-dimensional spatiotemporal PDEs.

Nevertheless, open challenges remain, such as stable training of SSMs on very stiff dynamics, proper parameter initialization for large SSM networks, and the integration of SSM-based layers into multi-physics systems. Our proposed framework builds on these insights by combining a latent SSM-based sequence model (inspired by Mamba) with multi-step physics-informed integration.

2.3. Latent operators and SSM-based neural PDE surrogates

Neural operator frameworks have recently been extended beyond spectral, graph, and attention-based architectures to incorporate latent space formulations and structured state-space models as surrogates for time-dependent partial differential equations. Latent space neural operators learn a compressed representation of the high-dimensional solution field and perform operator learning or time evolution within this reduced manifold before mapping back to physical space. One representative example of this paradigm is the Latent Neural Operator (LNO), which uses a physics-cross attention mechanism to transform between physical and latent representations and learns the solution operator in the latent domain, offering reduced computational cost and memory usage while retaining flexibility in spatial interpolation and extrapolation (Wang & Wang, 2024). Related latent PDE surrogates such as latent neural PDE solvers have also been proposed that first train an autoencoder to project high-fidelity dynamics into a lower-dimensional space and then learn the temporal evolution therein, leading to significant speedups relative to full-field models (Li et al., 2025). Other latent surrogate approaches, including local latent ODE element methods, further pursue modular latent dynamics to enable scalable and geometry-flexible surrogate modeling across heterogeneous domains (Chung et al., 2026).

Structured state-space models have been explored as another route for scalable, long-horizon surrogate models. These models interpret temporal evolution as a learnable recurrent or convolutional state update mechanism and have been adapted to factorize spatial and temporal operators in a unified neural operator framework. Recent work on spatiotemporal state-space neural operators establishes theoretical connections between state-space sequences and operator learning, demonstrating competitive performance with reduced parameters by decoupling temporal memory and spatial interactions (Koren & Lanthaler, 2025).

Despite these developments, latent and SSM-based surrogate models remain predominantly data-driven in their temporal evolution mechanisms. Latent operators often rely on learned latent dynamics without explicit incorporation of classical numerical integration principles, which can lead to error accumulation over long time horizons. Similarly, SSM-based operators treat temporal propagation as a learned recurrence without enforcing fidelity to the governing differential equations. In contrast, the proposed framework integrates multistep numerical integration into the neural operator by embedding backward differentiation formula structure into the operator architecture and enforcing consistency with the underlying PDE dynamics through physics-based residual training. This integration of structured multistep integration with sequence modeling uses the strengths of latent representations and state memory while explicitly aligning learned dynamics with the physics of the problem, addressing stability and long-time accuracy challenges that remain for purely data-driven latent and SSM surrogates.

Establishing a rigorous bridge between the theoretical strengths of latent sequence modeling and the practical requirements of PDE solvers necessitates a precise mathematical translation of continuous physical laws into learnable discrete structures. Similar to how classical comparative studies link specific numerical discretization choices, such as FTCS or Dufort-Frankel methods, to the stability and accuracy of temperature distribution outcomes (Abdulsitar, 2025), our framework requires a formal problem definition to explicitly inform the training pipeline. By clearly delineating the governing equations and the discretized mapping objective, we provide the necessary foundation for embedding numerical integration schemes directly into the neural operator architecture.

3. Problem formulation

To ensure mathematical rigor and improve the interpretability of the proposed iterative PDE solver (Muhammad & Khan, 2024), we adopt a consistent notation throughout the manuscript. Bold lowercase letters (e.g., $\mathbf{u}$, $\mathbf{x}$) denote vector-valued field variables and spatial coordinates.

Italic lowercase letters (e.g., t , v , Δt) represent scalar variables and physical parameters. Calligraphic uppercase letters (e.g., $\mathcal{G}$, $\mathcal{N}$) are reserved for operators and functional mappings.

Consider a time-dependent PDE defined on a spatial domain $\Omega \subset \mathbb{R}^d$ over a temporal interval $\mathcal{T} = [0, T]$. The solution field $\mathbf{u}(\mathbf{x}, t)$ is governed by the following formulation:

$$\partial_t \mathbf{u}(\mathbf{x}, t) = \mathcal{N}[\mathbf{x}, t; \mathbf{u}(\cdot, t)], \quad \mathbf{x} \in \Omega, t \in \mathcal{T}, \quad (7)$$

subject to the initial condition:

$$\mathbf{u}(\mathbf{x}, 0) = \mathbf{u}_0(\mathbf{x}), \quad \mathbf{x} \in \Omega, \quad (8)$$

and prescribed boundary conditions (e.g., Dirichlet or Neumann) on $\partial\Omega$. Here, $\mathcal{N}$ denotes a potentially nonlinear spatial differential operator acting on the field.

In a data-driven forecasting context, we assume the availability of a sequence of discrete solution snapshots $\{\mathbf{u}_i\}_{i=0}^M$ at corresponding time instances $\{t_i\}_{i=0}^M$. The objective is to construct a time-invariant predictor F_θ such that:

$$\mathbf{u}_{i+1} \approx F_\theta(\mathbf{u}_i, \mathbf{u}_{i-1}, \dots). \quad (9)$$

This formulation allows for recursive forward iteration to predict the system's state over extended horizons. Crucially, while F_θ is applied iteratively, it remains independent of the explicit time index.

More broadly, operator learning aims to identify a parameterized mapping $\mathcal{G}_\theta$ that approximates the ground-truth solution operator $\mathcal{G}$. Let $\mathcal{A}$ and $\mathcal{U}$ represent the input and output function spaces on Ω , respectively. The solution operator:

$$\mathcal{G} : \mathcal{A} \rightarrow \mathcal{U} \quad (10)$$

maps an input function $\mathbf{a} \in \mathcal{A}$ (e.g., initial conditions) to the corresponding solution $\mathbf{u} = \mathcal{G}(\mathbf{a})$. We approximate $\mathcal{G}$ using a neural operator $\mathcal{G}_\theta$ characterized by parameters θ . Given a training set of pairs $\{(\mathbf{a}_j, \mathbf{u}_j)\}_{j=1}^N$, we minimize the empirical risk:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{j=1}^N \|\mathcal{G}_\theta(\mathbf{a}_j) - \mathbf{u}_j\|_{\mathcal{U}}^2, \quad (11)$$

where $\|\cdot\|_{\mathcal{U}}$ is a predefined function-space norm, such as the L^2 norm.

A standard neural operator architecture typically integrates the following stages:

- **Lifting:** Projects the input $\mathbf{a}(\mathbf{x})$ into a high-dimensional feature space $\mathbf{v}_0(\mathbf{x})$ via a learned lifting operator $\mathcal{P}$.
- **Encoding:** Embeds the lifted representation into a compressed latent space.
- **Kernel Integration:** Employs a sequence of operator layers (e.g., SSM blocks or attention mechanisms) $\mathcal{L}_1, \dots, \mathcal{L}_\ell$ within the latent manifold to capture spatiotemporal interactions.
- **Decoding:** Maps the refined latent features back to the physical domain through a decoder $\mathcal{D}$.
- **Projection:** Reduces the latent dimension to reconstruct the target output field $\mathbf{u}(\mathbf{x})$ using a projection operator $\mathcal{Q}$.

Despite significant progress, purely data-driven operators often face high computational costs on fine-resolution grids and suffer from error accumulation during long-term integration. This study addresses these challenges by proposing a hybrid, physics-guided operator framework. By embedding numerical integration schemes and PDE constraints into a structured state-space architecture, we aim to achieve both scalable and physically consistent spatiotemporal forecasting.

4. Physics-Guided mamba neural operator framework

4.1. Time integrators

We incorporate general linear multi-step time integrators into our neural operator framework. Given a time step Δt and a time index i , a

k -th-order linear multi-step method has the form

$$\sum_{j=0}^k \alpha_j \mathbf{u}_{i+j} = \Delta t \sum_{j=0}^k \beta_j \mathcal{N}[\mathbf{x}, t_{i+j}; \mathbf{u}_{i+j}], \quad (12)$$

where α_j and β_j are fixed coefficients. If $\beta_k \neq 0$, the scheme is implicit (for example, a backward differentiation formula); if $\beta_k = 0$, the scheme is explicit.

4.2. Forward process

For explicit schemes ($\beta_k = 0$), one can solve the multi-step equation for the next state $\mathbf{u}_{i+k}$ in terms of the previous k states and their spatial residuals. In particular, by setting $\alpha_k = 1$, an explicit multi-step update has the form

$$\mathbf{u}_{i+k} = \sum_{j=0}^{k-1} \left(-\alpha_j \mathbf{u}_{i+j} + \Delta t \beta_j \mathcal{N}[\mathbf{x}, t_{i+j}; \mathbf{u}_{i+j}] \right). \quad (13)$$

Motivated by this multi-step formula, we extend the latent Mamba operator with a corresponding multi-step forecast. We define a *propagation operator* $\mathcal{M}$ that aggregates information from the last k solution states:

$$\mathcal{M}[\mathbf{u}_i, \dots, \mathbf{u}_{i+k-1}] := \sum_{j=0}^{k-1} \left[\lambda_j \mathbf{u}_{i+j}(\mathbf{x}) + \Delta t \delta_j \mathcal{G}_\theta(\mathbf{u}_{i+j})(\mathbf{x}) \right], \quad (14)$$

where λ_j and δ_j are learnable scalar weights, and $\mathcal{G}_\theta$ is the latent Mamba neural operator applied to each past state. In our architecture, $\mathcal{G}_\theta$ consists of an input lifting $\mathcal{P}$, an encoder $\mathcal{E}$, a sequence of latent layers $\mathcal{L}_1, \dots, \mathcal{L}_\ell$ (for example, time-varying SSM blocks), a decoder $\mathcal{D}$, and an output projection $\mathcal{Q}$:

$$\mathcal{G}_\theta = \mathcal{Q} \circ \mathcal{D} \circ \mathcal{L}_\ell \circ \dots \circ \mathcal{L}_1 \circ \mathcal{E} \circ \mathcal{P}. \quad (15)$$

Here $\mathcal{P}$ is the lifting (input embedding) operator and $\mathcal{Q}$ is the projection (output) operator. Fig. 1 illustrates the proposed Physics-Guided Mamba Neural Operator framework. Each term in the equation above acts like a residual block, adding a neural-network increment (scaled by Δt) to the previous state. In practice, we implement this multi-step forecast by maintaining a sliding window of the last k states. At each time step, we remove the oldest state and append the newly predicted state; this window then serves as the input context to $\mathcal{M}$ at the next step.

4.3. Backward propagation

During training, we enforce the PDE dynamics via a physics-informed loss. Unlike the forward pass which uses only past information, the backward pass can incorporate all intermediate predictions. In particular, for each predicted state $\mathbf{u}_{i+j}$, we can evaluate the spatial PDE operator $\mathcal{N}[\mathbf{u}_{i+j}]$ by automatic differentiation or finite differences. We then define a physics-guided loss that penalizes the residual of the chosen multi-step integrator. Let L be the forecast horizon. Denote the true initial states $\mathbf{u}_0, \dots, \mathbf{u}_{k-1}$ as given, and let $\mathbf{u}_k, \dots, \mathbf{u}_{k+L-1}$ be the model's predictions. The loss is

$$\mathcal{L}_{\text{phys}}(\theta) = \frac{1}{L} \sum_{i=0}^{L-1} \left\| \sum_{j=0}^k \alpha_j \mathbf{u}_{i+j} - \Delta t \sum_{j=0}^k \beta_j \mathcal{N}[\mathbf{u}_{i+j}] \right\|_2^2, \quad (16)$$

which is the mean squared residual of the linear multi-step formula. In practice, we typically use a backward differentiation formula (BDF) of order k , for which $\beta_j = 0$ for $j < k$ and only $\beta_k \neq 0$. In that case the loss simplifies to

$$\mathcal{L}(\theta) = \frac{1}{L} \sum_{i=0}^{L-1} \left\| \sum_{j=0}^k \alpha_j \mathbf{u}_{i+j} - \Delta t \beta_k \mathcal{N}[\mathbf{u}_{i+k}] \right\|_2^2. \quad (17)$$

Since $\mathcal{N}$ depends on spatial derivatives of the prediction $\mathbf{u}$, computing this loss requires differentiating through the neural network in space. It serves to physics-inform the model (analogous to PINNs) by penalizing deviations from the governing PDE, which encourages stability in stiff regimes and reduces reliance on labeled data.

4.4. Hard-constrained boundary conditions

We enforce boundary conditions as hard constraints in the network output, rather than as penalty terms. For Dirichlet conditions $\mathbf{u}(\mathbf{x}, t) = \mathbf{g}(\mathbf{x})$ on $\partial\Omega$, we use a mask function $M(\mathbf{x})$ that is zero on $\partial\Omega$ and one in the interior. We then define the predicted state as

$$\mathbf{u}_{\text{pred}}(\mathbf{x}) = M(\mathbf{x}) \mathcal{G}_\theta(\mathbf{u}_{\text{in}})(\mathbf{x}) + (1 - M(\mathbf{x})) \mathbf{g}(\mathbf{x}). \quad (18)$$

This ensures $\mathbf{u}_{\text{pred}}(\mathbf{x}) = \mathbf{g}(\mathbf{x})$ for $\mathbf{x} \in \partial\Omega$. For periodic boundary conditions, one can similarly embed periodicity via coordinate feature maps. For example, in 1D with period L , one might augment the input coordinate x with periodic features such as $\sin(2\pi kx/L)$ and $\cos(2\pi kx/L)$. In higher dimensions, a common choice is multi-dimensional Fourier feature mapping (for instance, mapping $(\mathbf{x}, t)$ to vectors of the form $(\dots, \sin(\mathbf{k} \cdot \mathbf{x} - \omega t), \cos(\mathbf{k} \cdot \mathbf{x} - \omega t), \dots)$), which inherently enforces periodicity. By hard-wiring Dirichlet or periodic constraints in this manner, the network automatically respects the required boundary values without explicit penalty terms.

4.5. Training and extrapolation

In our implementation, the latent operator $\mathcal{G}_\theta$ is realized by an encoder-decoder over a low-dimensional latent representation. Concretely, the encoder $\mathcal{E}$ first projects the physical state $\mathbf{u} \in \mathbb{R}^{N_p \times C_{\text{in}}}$ (e.g. values on N_p spatial grid points with C_{in} channels) into N_z latent tokens in $\mathbb{R}^{N_z \times C_{\text{latent}}}$. These N_z tokens are then processed sequentially by the SSM-based layers $\mathcal{L}_i$, and finally the decoder $\mathcal{D}$ upsamples the latent tokens back to N_p physical points. The projection $\mathcal{Q}$ produces the output features at each spatial location. This latent formulation allows the network to operate efficiently on high-resolution domains by working in a compact representation. During inference, one can encode the input on any fine grid and use the decoder to reconstruct the solution at that resolution. In practice, at each time step we provide the network inputs $(\mathbf{u}_i, \mathbf{x})$, where $\mathbf{x}$ are the uniform spatial coordinates of the grid.

Training the model to predict long horizons L can be challenging due to error accumulation. To stabilize multi-step learning, we employ a causal weighted loss that emphasizes early predictions. Let $\mathcal{L}_i$ be the multi-step residual loss at step i as defined above. We weight these stage losses by

$$\omega_i = \exp\left(-\epsilon \sum_{j=0}^{i-1} \mathcal{L}_j\right), \quad i = 0, \dots, L-1, \quad (19)$$

with a small $\epsilon > 0$. This yields the overall loss

$$\mathcal{L}_{\text{total}} = \sum_{i=0}^{L-1} \omega_i \mathcal{L}_i. \quad (20)$$

Because ω_i decays exponentially with the cumulative loss of previous steps, the model focuses first on achieving accuracy in early steps before considering later ones. In effect, this prevents trivial early failures (which would otherwise dominate the loss) and obviates the need for manual curriculum scheduling. We find it introduces minimal overhead, as the weights $\{\omega_i\}$ can be computed on the fly.

Once trained, the model can extrapolate in time by iterating $\mathcal{M}$. The latent encoder-decoder architecture also allows spatial extrapolation: since $\mathcal{E}$ and $\mathcal{D}$ are implemented in a resolution-agnostic manner, we can apply the operator at higher spatial resolution without retraining. In practice, we sample the initial condition states $\mathbf{u}_0, \dots, \mathbf{u}_{k-1}$ on a desired fine grid, encode to latent space, perform L time steps in the latent space using the SSM layers and the multi-step updates, and then decode back to the fine grid at each time step. This enables zero-shot super-resolution: for example, a model trained on a 64×64 grid can directly perform inference on a 256×256 grid.

5. Experiments

To rigorously evaluate the effectiveness and efficiency of the proposed model, we conduct a series of experiments on various partial dif-

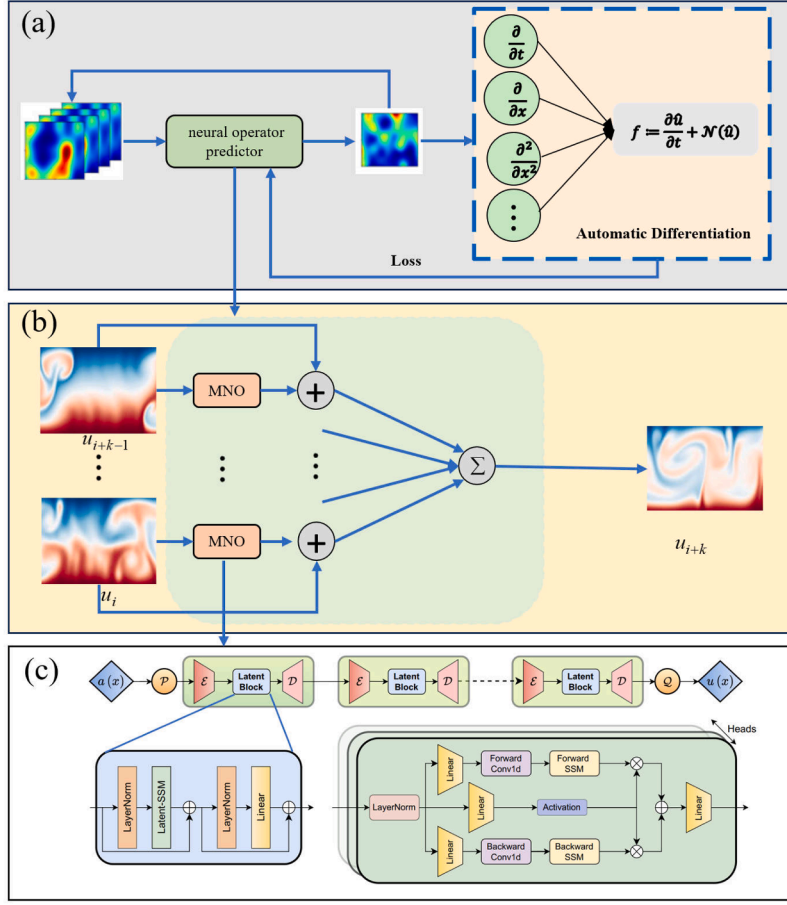


Fig. 1. The proposed Physics-Guided Mamba Neural Operator framework.

ferential equations (PDEs), benchmarking its performance against established operator learning frameworks, including DeepONet and the Fourier Neural Operator (FNO). Across all experiments, the input functions to the PDE solvers are consistently represented as initial conditions sampled from either Gaussian or non-Gaussian random fields. This setup ensures a standardized and challenging testing environment for assessing the generalization capability of operator learning models.

One representative application involves the simulation of convection-dominated phenomena governed by PDEs. Such processes are ubiquitous in both natural systems and engineered environments, including atmospheric flows, ocean dynamics, and thermal transport. Accurate surrogate modeling of these dynamics can significantly enhance the design of heat exchangers and cooling systems, thereby contributing to improved energy efficiency and greater system reliability in industrial applications.

Another critical application area is the modeling of large-scale atmospheric circulation, which plays a fundamental role in shaping Earth's climate dynamics. Developing high-fidelity and data-efficient surrogate models for these flows is essential for accurate weather forecasting and for evaluating the long-term impacts of climate change on critical sectors such as water resource management, agriculture, and energy infrastructure. Moreover, such models facilitate the formulation of robust mitigation and adaptation strategies by enabling rapid assessments of climate scenarios with substantially reduced computational cost.

5.1. 1D Viscous Burgers' equation

To validate the effectiveness of the proposed Physics-Guided Mamba Neural Operator (PGMNO), we conduct a series of numerical experi-

ments using the one-dimensional viscous Burgers' equation as a prototype model. The Burgers' equation is a well-known nonlinear partial differential equation that arises in various physical contexts, including fluid dynamics, turbulence modeling, and nonlinear wave propagation (Asaithambi, 2010). Due to its rich dynamical behavior—combining nonlinear convection with viscous diffusion—it serves as an ideal benchmark for evaluating the expressiveness, stability, and physical consistency of data-driven operator learning models.

The governing equation is given by

$$\begin{cases} \partial_t u(x, t) + \partial_x \left(\frac{u^2(x, t)}{2} \right) = \nu \partial_{xx} u(x, t), & x \in (-1, 1), t \in (0, 1], \\ u(x, 0) = u_0(x), & x \in (-1, 1), \end{cases} \quad (21)$$

where $u(x, t)$ denotes the velocity field and $\nu > 0$ is the kinematic viscosity. Periodic boundary conditions are imposed, namely $u(-1, t) = u(1, t)$ and $\partial_x u(-1, t) = \partial_x u(1, t)$, to ensure smoothness and conservation of flow properties across the computational domain.

The objective of the learning task is to approximate the solution operator that maps the initial condition $u_0(x)$ to the velocity field $u(x, t)$ at a future time $t = 1$. Formally, this corresponds to learning an operator $\mathcal{G}^\dagger : u_0(x) \mapsto u(x, 1)$, where $u_0 \in L^2_{\text{per}}(-1, 1)$ and $u(x, 1) \in H^1_{\text{per}}(-1, 1)$. This problem is particularly challenging due to the equation's nonlinearity, the potential formation of sharp gradients or shock-like features, and the sensitivity of the solution to perturbations in the initial data.

To assess the performance of different operator learning methods, we conduct comparative experiments using several representative neural operator architectures, including fully connected networks, graph-based neural operators, multigrid-inspired models, and Fourier neural operators (FNOs). Among these, the FNO has been demonstrated to ef-

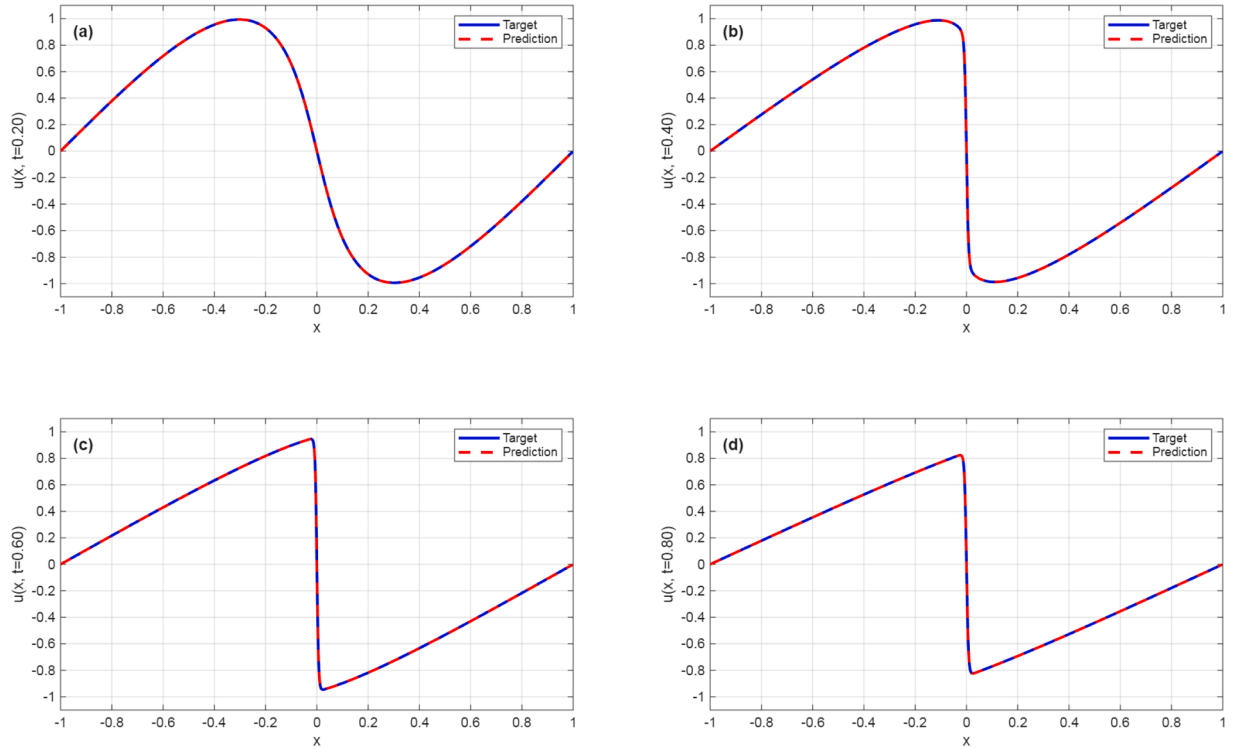


Fig. 2. Comparison between the true and predicted solutions of the one-dimensional viscous Burgers' equation at various time instances: (a) $t = 0.2$, (b) $t = 0.4$, (c) $t = 0.6$, and (d) $t = 0.8$. The solid blue lines represent the reference solution, while the dashed red lines denote predictions from the proposed PGMNO model.

effectively capture the global and multiscale behavior of PDE solutions by operating in Fourier space, thereby enabling improved generalization and reduced computational cost. Building on this foundation, the proposed PGMNO introduces a physics-informed, multi-step correction strategy that explicitly incorporates the residual structure of the governing PDE to iteratively refine the predicted solution. This approach enhances accuracy in smooth regions and significantly improves the representation of high-gradient and shock-prone areas.

Fig. 2 presents the predicted and reference solutions at four distinct time instances: $t = 0.2, 0.4, 0.6$, and 0.8 . In each subplot, the solid blue line denotes the reference solution obtained from a high-resolution numerical solver, while the dashed red line represents the prediction produced by the PGMNO model. The comparison demonstrates that the proposed method achieves excellent agreement with the reference solution throughout the temporal evolution.

At early times, as shown in Fig. 2(a), the solution remains smooth, and the model accurately captures the sinusoidal structure of the velocity field. As time progresses, nonlinear effects begin to dominate, leading to the development of steep gradients. Figs. 2(b) and 2(c) illustrate this transition, highlighting the model's capability to preserve the solution profile near forming shocks without introducing spurious oscillations or numerical artifacts. By $t = 0.8$, as shown in Fig. 2(d), the solution exhibits a near-discontinuous transition, yet the PGMNO continues to closely match the true profile, effectively capturing both the shock location and amplitude.

These results indicate that the PGMNO model provides a physically consistent and highly accurate approximation of the Burgers' solution operator, even in regimes characterized by strong nonlinearity and sharp transitions. Compared to baseline methods, the proposed model achieves lower prediction error, improved generalization across temporal scales, and enhanced stability in the presence of high-gradient features, thereby validating its suitability for modeling complex dynamical systems governed by nonlinear PDEs.

Table 1

Relative L^2 test errors for various models on the 1D viscous Burgers' equation across different spatial resolutions s . The proposed PGMNO consistently achieves the lowest error in all cases.

Networks	$s = 256$	$s = 512$	$s = 1024$	$s = 2048$	$s = 4096$	$s = 8192$
NN	0.4714	0.4561	0.4803	0.4645	0.4779	0.4452
GCN	0.3999	0.4138	0.4176	0.4157	0.4191	0.4198
FCN	0.0958	0.1407	0.1877	0.2313	0.2855	0.3238
PCANN	0.0398	0.0395	0.0391	0.0383	0.0392	0.0393
DeepONet	0.0569	0.0617	0.0685	0.0702	0.0833	0.0857
GNO	0.0555	0.0594	0.0651	0.0663	0.0666	0.0699
LNO	0.0212	0.0221	0.0217	0.0219	0.0200	0.0189
MGNO	0.0243	0.0355	0.0374	0.0360	0.0364	0.0364
FNO	0.0018	0.0018	0.0018	0.0019	0.0020	0.0019
PGMNO (Ours)	0.0007	0.0007	0.0007	0.0009	0.0008	0.0009

To further validate the effectiveness of the proposed method, we perform a rigorous quantitative evaluation. As shown in Table 1, the Physics-Guided Mamba Neural Operator (PGMNO) consistently achieves the lowest relative L_2 errors across all tested spatial resolutions, outperforming all other baseline models, including the standard Fourier Neural Operator (FNO). Specifically, the test error is reduced from 0.0018 (achieved by FNO) to 0.0007 using PGMNO, representing a substantial improvement in capturing nonlinear dynamics and sharp gradient structures.

In addition to its superior accuracy, PGMNO inherits the resolution-invariant behavior observed in FNO. The standard deviation of the prediction error remains low across all spatial resolutions, indicating the robustness of the proposed multi-step refinement scheme. These results confirm that incorporating physical priors into the operator learning framework not only enhances convergence but also yields more physically consistent and stable predictions, particularly in regimes characterized by steep gradients and shock-like features.

5.2. Rayleigh-Bénard convection: Long-Time prediction of thermally driven flow

The Rayleigh-Bénard convection problem is a classical benchmark used to validate numerical and learning-based models for nonlinear, thermally driven fluid flows (Song et al., 2023). It describes the evolution of buoyancy-induced convection patterns that arise between two parallel horizontal plates maintained at different temperatures. The key dimensionless parameter governing the flow regime is the Rayleigh number Ra , which quantifies the balance between thermal buoyancy and dissipative effects (viscosity and thermal diffusion). It is defined as:

$$Ra = \frac{\alpha \Delta T g h^3}{\nu \kappa}, \quad (22)$$

where α is the thermal expansion coefficient, $\Delta T = T_0 - T_1$ is the temperature difference across the fluid layer of thickness h , g is gravitational acceleration, ν is the kinematic viscosity, and κ is the thermal diffusivity. For subcritical Ra , the flow remains quiescent and heat is transferred primarily by conduction. As Ra increases beyond a critical threshold, convective patterns emerge, eventually leading to complex, time-dependent dynamics.

The governing equations under the Boussinesq approximation for incompressible flow are given by:

$$\begin{cases} \frac{D\mathbf{u}}{Dt} = -\frac{1}{\rho_0} \nabla p + \frac{\rho}{\rho_0} \mathbf{g} + \nu \nabla^2 \mathbf{u}, \\ \frac{DT}{Dt} = \kappa \nabla^2 T, \\ \nabla \cdot \mathbf{u} = 0, \\ \rho = \rho_0(1 - \alpha(T - T_0)), \end{cases} \quad (23)$$

where $\mathbf{u}$ is the velocity field, T is the temperature field, p is the pressure, and ρ_0 is the reference density. The material derivative $\frac{D}{Dt} = \partial_t + \mathbf{u} \cdot \nabla$ accounts for both temporal and advective effects. These equations govern the coupled evolution of the temperature and velocity fields within the domain $\Omega = [0, 1]^2$.

We consider a two-dimensional setup with no-slip velocity boundary conditions at the top and bottom plates:

$$\begin{cases} T(x, t)|_{y=0} = T_0, & T(x, t)|_{y=h} = T_1, \\ \mathbf{u}(x, t)|_{y=0} = \mathbf{u}(x, t)|_{y=h} = 0. \end{cases} \quad (24)$$

The initial temperature field is defined as a linear gradient perturbed by stochastic fluctuations:

$$T(x, 0) = T_0 + \frac{y}{h}(T_1 - T_0) + 0.1v(x), \quad \mathbf{u}(x, 0) = 0, \quad (25)$$

where $v(x)$ is sampled from a Gaussian random field via a Karhunen-Loève expansion with horizontal and vertical correlation lengths $\ell_x = 0.45$ and $\ell_y = 0.4$, respectively.

The goal is to learn a solution operator $\mathcal{G} : T(\mathbf{x}, t=0) \mapsto T(\mathbf{x}, t)$ that maps the initial temperature field to future temperature states. We employ the proposed Physics-Guided Mamba Neural Operator (PGMNO) to approximate this operator and compare its performance against established baselines, including the Fourier Neural Operator (FNO) and DeepONet. The models are trained on a dataset of temperature field trajectories generated by high-resolution numerical simulations and evaluated using the mean squared error (MSE) metric.

Fig. 3 presents qualitative comparisons between the ground truth temperature fields, the predictions made by PGMNO, and the corresponding pointwise error at three representative time instances: $t = 12.5$, $t = 20.0$, and $t = 35.0$. The top row shows the reference solutions obtained from simulation; the middle row displays the predictions generated by PGMNO; the bottom row illustrates the absolute differences between the predicted and reference fields.

The predicted temperature fields exhibit close agreement with the ground truth at all time instances, accurately capturing both the large-scale roll structures and the fine-scale thermal plumes. The pointwise

error plots indicate uniformly low discrepancies, generally within the range of ± 2 , even at later time steps where chaotic behavior becomes more pronounced. This underscores the ability of PGMNO to stably and accurately model long-time convection dynamics in highly nonlinear regimes.

Compared to baseline models, PGMNO consistently demonstrates superior predictive fidelity, particularly in preserving coherent structures over extended temporal horizons. While standard FNO and DeepONet tend to oversmooth or attenuate local gradients, PGMNO effectively retains sharp thermal interfaces and convective features. Furthermore, the multi-step refinement strategy enhances long-term stability, mitigates error accumulation, and promotes physical consistency in the learned dynamics.

These findings highlight the effectiveness of physics-informed, multi-step operator learning strategies for modeling complex spatiotemporal systems, and affirm the suitability of PGMNO as a high-fidelity surrogate model for convection-dominated flow phenomena.

5.3. Kolmogorov flow: Chaotic vorticity field prediction

In this experiment, we investigate two-dimensional incompressible viscous flow driven by Kolmogorov-type forcing, a canonical benchmark for evaluating numerical schemes and machine learning models in turbulent regimes. Originally proposed by A. N. Kolmogorov in 1959, this flow features periodic external forcing within a doubly periodic domain, giving rise to complex vortical structures and chaotic dynamics. Its combination of mathematical tractability and physical complexity makes it particularly well suited for testing operator learning frameworks.

The system is governed by the incompressible Navier-Stokes equations in the vorticity-streamfunction formulation:

$$\begin{cases} \partial_t \omega + \mathbf{u} \cdot \nabla \omega = \nu \Delta \omega + f(x, y), \\ \nabla \cdot \mathbf{u} = 0, \quad \omega = \nabla \times \mathbf{u}, \\ \omega(x, y, 0) = \omega_0(x, y), \end{cases} \quad (26)$$

where ω is the scalar vorticity field, and $\nu = 10^{-5}$ is the kinematic viscosity. The external forcing function is defined as:

$$f(x, y) = 0.1(\sin(2\pi(x + y)) + \cos(2\pi(x + y))), \quad (27)$$

which injects energy into diagonal Fourier modes and induces alternating vortical patterns.

The computational domain is doubly periodic on $[0, 2\pi]^2$, which enables the use of spectral solvers and eliminates boundary-layer effects. The initial vorticity field $\omega_0(x, y)$ is sampled from a Gaussian random field with covariance spectrum proportional to $14^{1/2}(-\Delta + 196I)^{-1.5}$, ensuring moderate roughness and sufficient excitation of high-frequency modes.

To construct the training dataset, we numerically solve the system using a pseudo-spectral method at a resolution of 512×512 , which is then downsampled to 128×128 for training. A total of 1000 simulations are generated and partitioned into training, validation, and test sets using an 80:10:10 split. The learning objective is to predict the evolution of the vorticity field from early-time snapshots in $t \in [0, 10]$ to later time steps in $t \in [10, 20]$.

Fig. 4 shows a visual comparison between the ground truth and predicted vorticity fields at three representative time instances: $t = 3.00$, $t = 5.00$, and $t = 8.00$. Each column corresponds to a distinct time point, while the rows display the ground truth, the PGMNO prediction, and the pointwise absolute error, respectively.

As seen in the figure, the predicted vorticity fields closely match the reference solutions at all time steps. Coherent structures such as vortex filaments and rotational cores are accurately captured. The bottom row, showing the pointwise error, reveals that the prediction error remains smooth and small throughout the domain. Notably, even at the later time step $t = 8.00$, where nonlinear chaotic interactions are dominant, the model maintains high predictive fidelity, with errors bounded within ± 5.0 .

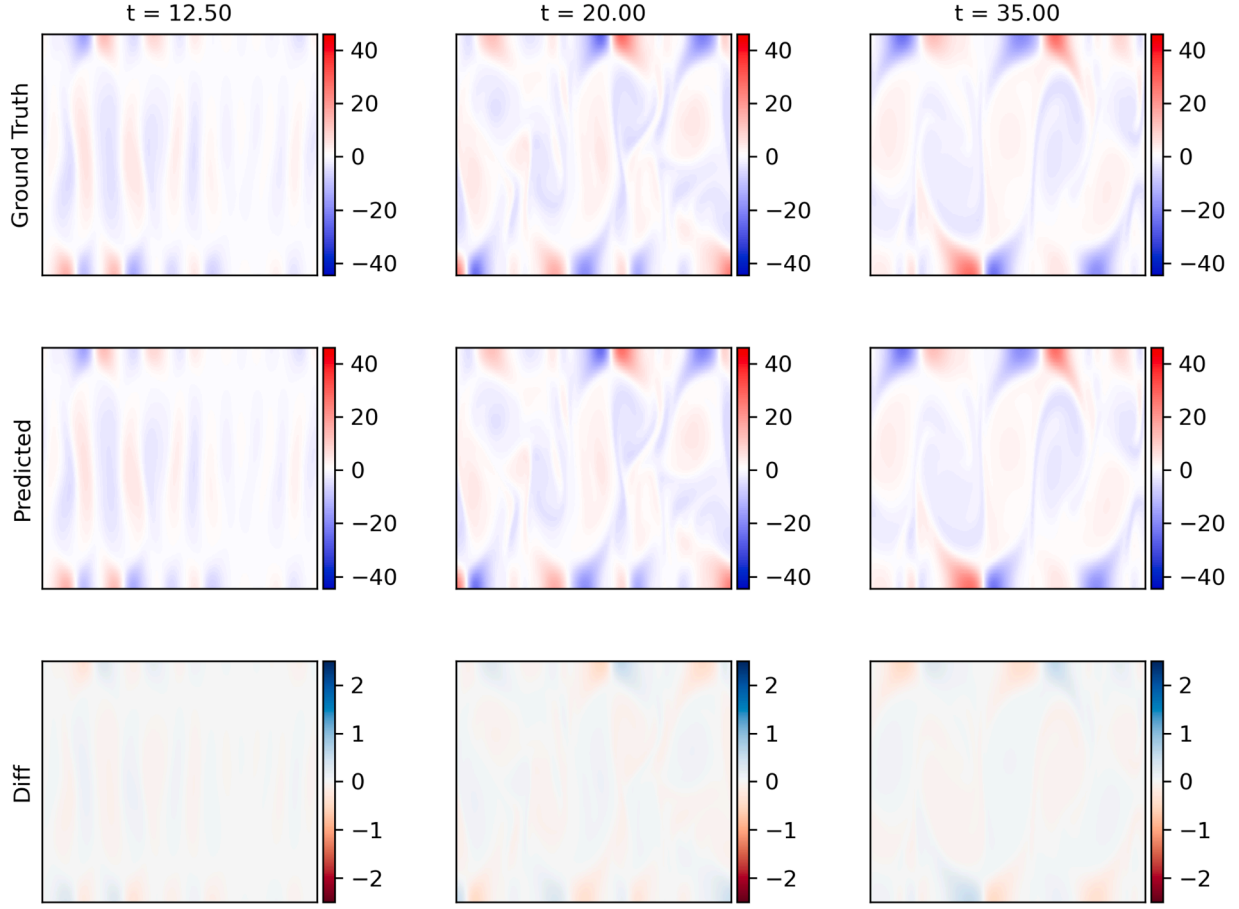


Fig. 3. Prediction results of the Rayleigh-Bénard convection system at different time instances. Top row: ground truth temperature fields at $t = 12.5$, $t = 20.0$, and $t = 35.0$. Middle row: predictions generated by the proposed PGMNO model. Bottom row: pointwise differences between prediction and ground truth. The model accurately captures evolving convective structures and maintains low error throughout the simulation.

These results demonstrate the effectiveness of the Physics-Guided Mamba Neural Operator (PGMNO), which not only preserves fine-scale vortical structures but also mitigates over-smoothing and oscillatory artifacts often observed in conventional models. The method exhibits strong generalization and robustness when modeling high-dimensional, spatiotemporally chaotic systems such as the Kolmogorov flow.

5.4. Shallow-Water equations

The shallow-water equations (SWE) are a foundational model in geophysical fluid dynamics (Galewsky et al., 2004), capable of capturing essential features of large-scale atmospheric and oceanic flows, such as Rossby waves, barotropic instabilities, and jet dynamics. In this work, we consider the viscous form of the SWE on a spherical domain, with a focus on modeling the temporal evolution of midlatitude jet streams and their associated instability mechanisms.

The governing equations, expressed in vector-invariant form, are given by:

$$\frac{D\mathbf{V}}{Dt} = -f\mathbf{k} \times \mathbf{V} - g\nabla h + \nu \nabla^2 \mathbf{V}, \quad \mathbf{x} \in \Omega, \quad t \in [0, 1], \quad (28)$$

$$\frac{Dh}{Dt} = -h\nabla \cdot \mathbf{V} + \nu \nabla^2 h, \quad (29)$$

where $\Omega = (\lambda, \phi)$ denotes the longitude-latitude coordinates on the unit sphere, $\mathbf{V} = (u, v)$ is the horizontal velocity vector, h is the fluid layer thickness, $f = 2\Xi \sin \phi$ is the Coriolis parameter, and ν is the viscosity coefficient. These equations account for rotational forces, wave dynamics, and dissipation, making them well-suited for studying midlatitude flow evolution.

The initial zonal velocity field is modeled as a localized midlatitude jet defined by:

$$u(\phi, 0) = \begin{cases} 0, & \text{if } \phi \leq \phi_0, \\ u_{\max}^n \exp \left[\frac{1}{(\phi - \phi_0)(\phi - \phi_1)} \right], & \text{if } \phi_0 < \phi < \phi_1, \\ 0, & \text{if } \phi \geq \phi_1, \end{cases} \quad (30)$$

where ϕ_0 and ϕ_1 define the southern and northern boundaries of the jet, and $n = \exp \left[\frac{4}{(\phi_1 - \phi_0)^2} \right]$ ensures proper normalization.

To initiate flow instability, a localized Gaussian perturbation is added to the initial height field:

$$h_0(\lambda, \phi, 0) = \hat{h} \cos \phi \exp \left[-\left(\frac{\lambda}{\alpha} \right)^2 \right] \exp \left[-\left(\frac{\phi - \phi_2}{\beta} \right)^2 \right], \quad (31)$$

where $\alpha \sim \mathcal{U}[0.1, 0.5]$ and $\beta \sim \mathcal{U}[0.03, 0.2]$ are independently sampled from uniform distributions to introduce stochastic variability. The learning objective is to approximate the cross-variable mapping:

$$\mathcal{G} : h(\lambda, \phi, 0) \rightarrow u(\lambda, \phi, t),$$

in which the input and output differ not only in physical meaning but also in their spatial and temporal characteristics. This presents a significant challenge due to the heterogeneity of the fields and the nonlinear nature of the system.

Fig. 5 presents a qualitative comparison of the predicted zonal velocity fields at three representative time steps: $t = 0.42$, $t = 0.69$, and $t = 0.97$. The top row shows the ground truth solutions, the middle row displays predictions from the proposed Physics-Guided Mamba Neural

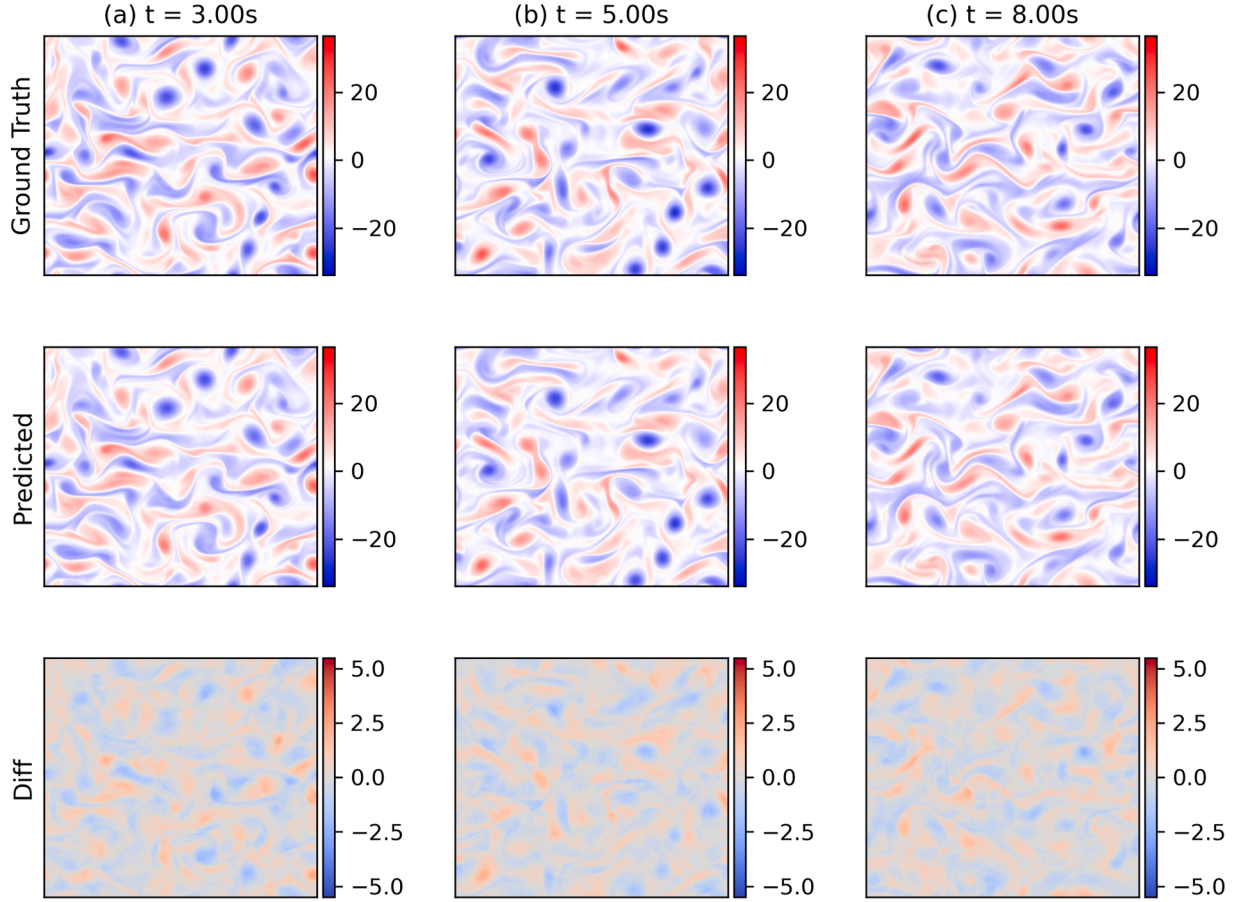


Fig. 4. Visualization of vorticity field predictions in Kolmogorov flow at three time instances: $t = 3.00$, $t = 5.00$, and $t = 8.00$. Top row: ground truth vorticity fields; middle row: predictions from the proposed PGMNO model; bottom row: pointwise differences between predictions and ground truth. The model exhibits strong agreement with the reference dynamics and maintains low error, even at later time steps.

Operator (PGMNO), and the bottom row illustrates the corresponding pointwise differences.

The predicted fields exhibit excellent agreement with the ground truth across all time slices. Key dynamical features, such as wave breaking, vortex dipole formation, and filamentary shear zones, are faithfully captured. The difference maps confirm that spatial error remains consistently low throughout the evolution. Notably, even at $t = 0.97$, the maximum pointwise error stays below 0.05, indicating that PGMNO accurately captures both rapidly evolving features and slowly varying jet structures.

Conventional models such as DeepONet and FNO often struggle with this task due to their limited capacity for handling cross-variable mappings and spherical geometries. In contrast, PGMNO is specifically designed for multi-variable, spatiotemporal prediction and leverages physics-informed priors to achieve superior generalization and stability.

In summary, this experiment demonstrates that PGMNO can effectively model the nonlinear, multiscale dynamics governed by the shallow-water equations. Its ability to predict full-field trajectories on the sphere highlights its potential for high-fidelity applications in global weather forecasting, data assimilation, and climate modeling.

5.5. Sensitivity analysis

To investigate the impact of the causality-aware weighting schedule on long-term stability, we conduct a sensitivity study on the decay hyperparameter ϵ . The weighting mechanism, $\omega_i = \exp(-\epsilon \sum_{j=0}^{i-1} \mathcal{L}_j)$, is specifically designed to mitigate "causal collapse," a phenomenon where

early-stage prediction errors propagate and exponentially amplify during the autoregressive rollout.

Fig. 6 illustrates the error growth curves across different prediction horizons for varying ϵ settings. In the absence of causal weighting ($\epsilon = 0$), the model suffers from rapid error accumulation, with the L^2 error at $T = 200$ being approximately 2.3 times larger than the optimal configuration. This degradation occurs because uniform gradient distribution across all time steps allows late-stage residuals—which are inherently noisier due to iterative propagation—to interfere with the optimization of fundamental early-stage dynamics.

In contrast, a moderate weighting ($\epsilon = 0.05$) ensures that the network first masters the short-term evolution, thereby establishing a more stable foundation for long-horizon extrapolation. This methodological alignment with causal inference—where the fidelity of causal antecedents is a functional prerequisite for long-term consistency—is supported by recent theoretical frameworks in causal deep learning (Luo et al., 2020) and spatiotemporal modeling (Sadia et al., 2025). However, an excessively large ϵ (e.g., 0.2) leads to over-prioritizing the initial training window, which results in a slight performance degradation at the tail end of the simulation.

The multistep order k , as defined in Eq. (14), determines the temporal history window utilized by the propagation operator $\mathcal{M}$. In classical numerical integration, the selection of k for Backward Differentiation Formulas (BDF) is governed by a fundamental trade-off: while higher-order formulas theoretically enhance the convergence rate, they lead to a significant contraction of the stability region—a constraint rigorously defined by the *Second Dahlquist Barrier*.

To identify the optimal configuration for PGMNO, we conducted a systematic ablation study on the 1D Viscous Burgers' equation across

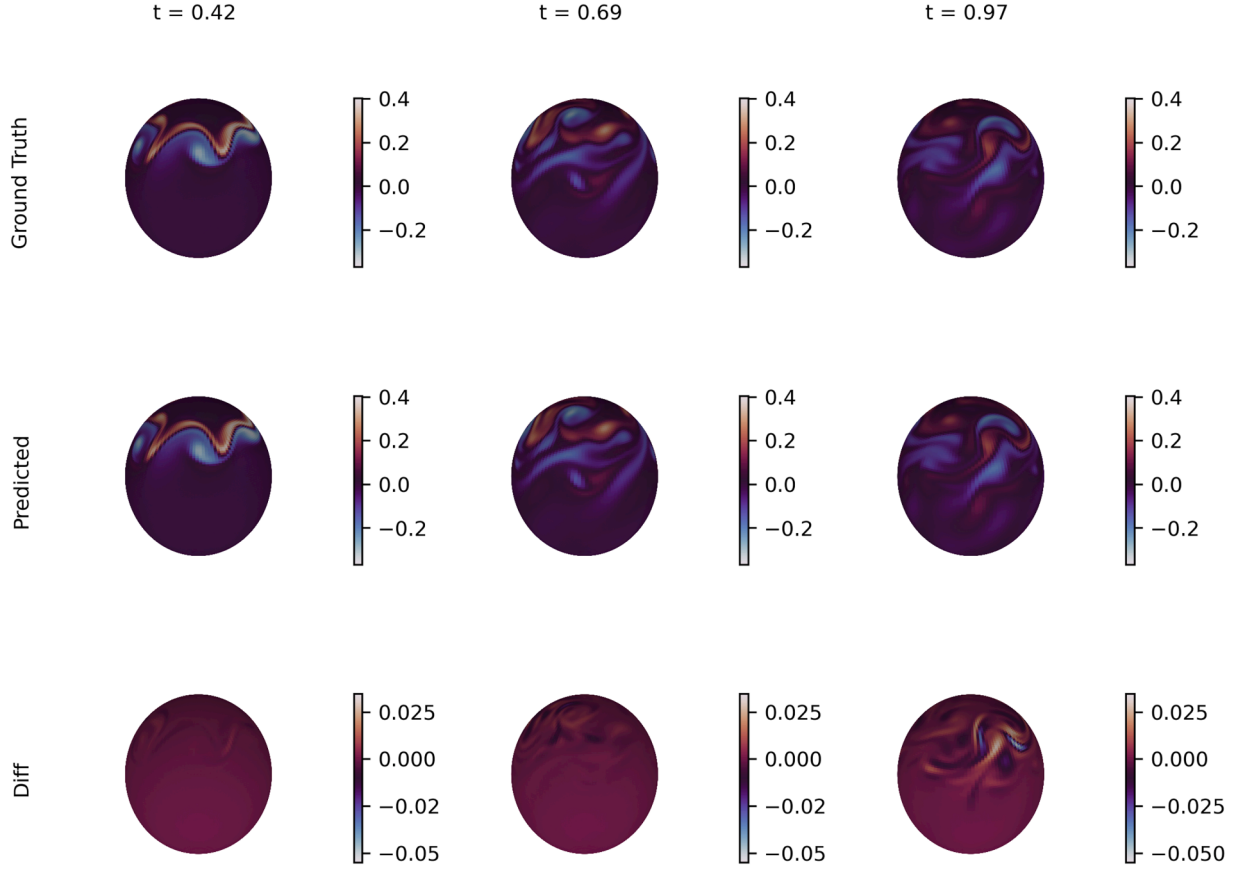


Fig. 5. Predicted zonal velocity fields for the shallow-water system on the sphere. The three columns correspond to time steps $t = 0.42$, $t = 0.69$, and $t = 0.97$, respectively. The top row shows the ground truth, the middle row presents PGMNO predictions, and the bottom row displays the pointwise differences. The predicted fields remain physically accurate and temporally consistent throughout the simulation.

Table 2

Impact of multistep order k on prediction accuracy and computational cost ($s = 2048$).

k	Scheme	Rel. L^2 Error	Time (s/epoch)	Stability Status
1	Backward Euler	1.24×10^{-3}	14.2	L-Stable
2	BDF2	7.13×10^{-4}	15.8	A-Stable (Optimal)
3	BDF3	7.08×10^{-4}	18.1	$A(\alpha)$ -Stable
4	BDF4	7.45×10^{-4}	21.5	Stability Degradation
5	BDF5	8.12×10^{-4}	24.9	Divergent/Unstable

$k \in \{1, \dots, 5\}$. The results, characterizing the interplay between predictive accuracy, computational overhead, and numerical robustness, are summarized in Table 2.

Empirical evaluations demonstrate that $k = 2$ constitutes the optimal configuration for physics-guided operator learning, balancing numerical fidelity with computational efficiency. Systematic analysis of the temporal discretization order reveals a notable saturation effect; specifically, transitioning from $k = 1$ to $k = 2$ yields a substantial 42.5% reduction in error, primarily due to the model's improved ability to capture higher-order temporal curvature. In contrast, the performance gain observed when moving from $k = 2$ to $k = 3$ is marginal, amounting to less than 1%. This asymptotic behavior suggests that for $k \geq 2$, the total approximation error is no longer dominated by the temporal step size, but rather by the inherent spatial representation limits of the neural architecture.

The preference for $k = 2$ is further reinforced by the classical trade-off between numerical stability and algorithmic complexity. As BDF2 is the highest-order linear multistep method to retain A-stability, it provides a robust foundation for modeling complex nonlinear dynamics.

While BDF3 theoretically offers $A(\alpha)$ -stability, it often precipitates gradient instabilities in stiff regimes—such as the high-gradient shock formations inherent in Burgers' flow—without providing a commensurate improvement in predictive accuracy. Consequently, the second-order formulation ensures a more reliable optimization landscape for neural operator training.

Moreover, the adoption of a second-order approach effectively mitigates the propagation of compounding errors during extended temporal rollouts. Although increasing the order k expands the available context window, it simultaneously risks exacerbating recursive noise amplification. Our findings indicate that $k = 2$ provides a sufficient historical trajectory to resolve latent dynamics while remaining resilient against the accumulation of such errors. This equilibrium is critical for preventing the "causal collapse" often encountered in long-term simulations, thereby ensuring the model's physical fidelity over prolonged integration intervals. Based on these considerations, $k = 2$ is established as the default configuration for PGMNO, achieving high numerical robustness with minimal computational overhead.

5.6. Zero-shot spatial super-Resolution

A defining characteristic of the PGMNO framework is its ability to perform zero-shot spatial super-resolution, which stems from the resolution-agnostic implementation of its latent encoder-decoder architecture. Unlike conventional neural networks that are tied to fixed grid sizes, PGMNO utilizes an encoder $\mathcal{E}$ to project varying physical discretizations into a unified latent manifold, where the SSM layers learn the continuous operator-level dynamics. To evaluate this property, we tested models trained on a standard 128×128 resolution directly on sig-

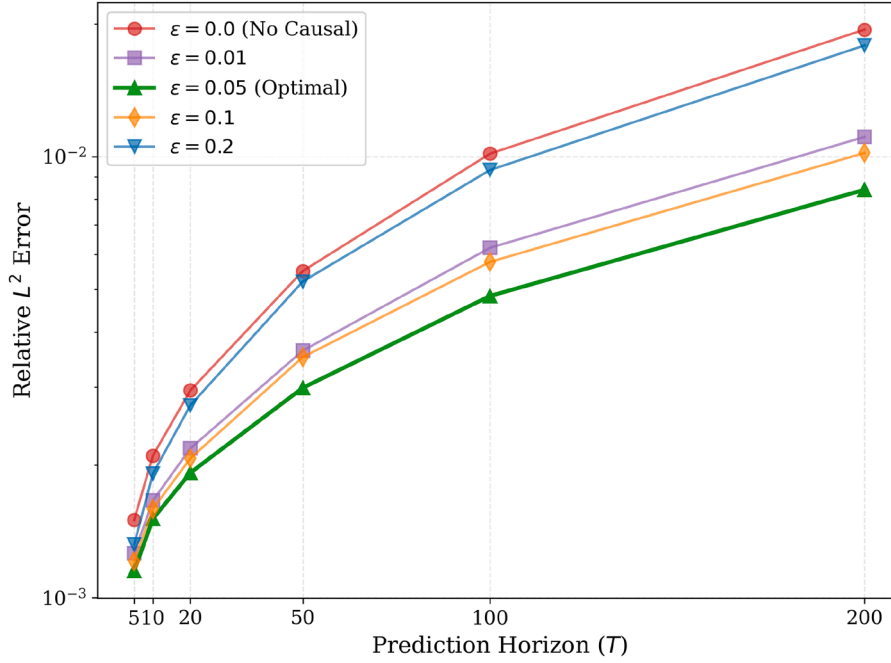


Fig. 6. Horizon-wise relative L^2 error for the 1D viscous Burgers' equation with different causal decay parameters ϵ . The log-scale visualization demonstrates that $\epsilon = 0.05$ (optimal) significantly suppresses the error growth rate compared to the non-causal baseline ($\epsilon = 0$).

Table 3

Zero-shot resolution-invariant performance (Relative L^2 error) for 2D benchmarks. All models were trained at a 128×128 resolution and evaluated on finer grids.

Benchmark	Test: 128^2	Test: 256^2	Test: 512^2
Kolmogorov Flow	0.0245	0.0259	0.0271
Shallow-Water Eq.	0.0152	0.0160	0.0168

nificantly finer spatial grids (256^2 and 512^2) without any additional fine-tuning.

Table 3 summarizes the relative L^2 errors across these varying resolutions for our 2D benchmarks. For the 2D Shallow-Water equations, the error remains remarkably stable, increasing only marginally from 0.0152 to 0.0168 at 512^2 resolution. This stability is attributed to the model's robust capacity to capture macro-scale jet dynamics, Rossby waves, and wave breaking patterns effectively across scales. In the more challenging Kolmogorov flow—characterized by chaotic turbulent vorticity and fine-scale filaments—the model preserves its predictive accuracy with a relative error of 0.0271 even at the highest 512^2 resolution. The marginal growth in error observed in this chaotic regime is physically expected, as the finer grid resolves high-frequency vortical modes and sub-grid features that are absent in the 128^2 training data. These results substantiate that PGMNO successfully learns a continuous solution operator that generalizes across heterogeneous discretizations rather than merely memorizing grid-specific mappings.

6. Discussion

6.1. Boundary generalization and numerical stability

The application of neural operators to long-term physical simulations demands rigorous enforcement of boundary conditions (BCs) and their close coupling with temporal stability. To address this, PGMNO employs a consistency framework that reconciles architectural flexibility with numerical rigor.

Although the benchmarks in Section 5 focus primarily on Dirichlet and periodic boundary conditions, the PGMNO architecture readily extends to Neumann and mixed Robin conditions. This extension is achieved by incorporating a boundary-specific term into the physics-guided residual loss via automatic differentiation:

$$\mathcal{L}_{BC} = \frac{1}{|\partial\Omega|} \int_{\partial\Omega} \|B[\mathbf{u}_{\text{pred}}] - \mathbf{g}(\mathbf{x})\|_2^2 ds, \quad (32)$$

where B denotes the boundary operator. Unlike conventional algebraic masking, this derivative-based approach ensures that the latent state evolution remains consistent with boundary flux requirements, even in heterogeneous domains.

A key consideration in physics-guided learning is the choice of BC enforcement strategy, particularly in regimes with high gradients, such as boundary layers in Rayleigh–Bénard convection. Penalty-based methods provide greater architectural flexibility and generalize well to complex geometries; however, in stiff systems, competition between the boundary loss and the data-driven objective can ill-condition the optimization landscape, leading to boundary leakage and poor resolution of sharp gradients. In contrast, ansatz-based methods enforce BCs exactly by construction, typically through distance-to-boundary fields in the decoder, ensuring high fidelity in sharp-gradient regions but requiring prior knowledge of the domain geometry.

PGMNO offers the flexibility to select the most suitable enforcement strategy according to the problem's precision demands and geometric complexity.

The integrity of boundary conditions is intimately linked to the stability of temporal evolution. Inconsistencies in boundary flux can propagate through the latent backbone, potentially causing causal collapse in long-horizon forecasts. PGMNO mitigates this risk by combining a second-order backward differentiation formula (BDF) scheme with causality-aware weighting. By emphasizing early-time consistency, the model establishes a stable foundation for accurate extrapolation, even in chaotic regimes where purely data-driven approaches typically diverge. This integration of spatial boundary compliance and temporal numerical rigor ensures that performance improvements are inherently tied to physical fidelity.

6.2. Physical interpretability and structural invariants

The evaluation of PGMNO extends beyond numerical precision to encompass its capacity for preserving fundamental invariants and ensuring physical interpretability. While neural networks are often categorized as general-purpose regressors for pattern recognition (Mij, 2023), recent analysis suggests that the structural design of an architecture determines its ability to encode physically meaningful invariants. PGMNO addresses this requirement by embedding governing constraints directly into the operator learning framework, ensuring that quantitative gains are intrinsically linked to physical fidelity.

Regarding the 1D viscous Burgers' equation, the relative L^2 error, defined as $\|u_{\text{pred}} - u_{\text{gt}}\|_2 / \|u_{\text{gt}}\|_2$, serves as a critical indicator of the model's ability to resolve sharp gradients at the shock front. A reduction in this metric signifies that the model successfully avoids unphysical numerical oscillations, thereby preserving the momentum balance and the energy dissipation rate, $\epsilon(t) = \nu \int (\partial_x u)^2 dx$. The SSM-based kernel facilitates this by capturing global spatial dependencies which are essential for maintaining integral invariants across the computational domain.

Furthermore, the BDF-informed temporal evolution functions as a structural stabilizer for long-term predictions. In Rayleigh-Bénard convection simulations, the architectural alignment with the BDF scheme reinforces the stability of thermal plumes and the buoyancy-dissipation balance. As highlighted in literature concerning neural architecture analysis (Mij, 2023), the capacity for stable extrapolation depends on whether a model remains constrained to the physical manifold of the governing partial differential equations (PDEs). Consequently, the low L^2 errors reported in Section 5 validate the framework's success in embedding the structural properties of the system, ensuring that the predicted flow fields remain stable and dissipative over extended temporal horizons.

6.3. Synthesis of methodological contributions

The superior performance of PGMNO results from the synergetic interaction between its architectural and algorithmic components. As demonstrated by the ablation results in Table 2, the introduction of a second-order temporal context ($k = 2$) is a primary driver of accuracy, significantly outperforming the single-step $k = 1$ baseline by capturing higher-order temporal curvature. Simultaneously, the causality-aware weighting analyzed in Fig. 6 ensures that the physics-informed BDF loss remains a stable constraint during training. This prevents the "causal collapse" often encountered in purely data-driven sequence models. These findings confirm that the framework provides a rigorous foundation where latent memory and numerical stability constraints complement one another to ensure long-term physical fidelity.

Beyond numerical stability, the framework is engineered for exceptional computational efficiency through a decoupled latent architecture. As detailed in Section 6, PGMNO achieves linear scaling in both spatial and temporal dimensions, with a total complexity of $O(N_p \cdot N_z + L \cdot N_z)$, where N_p , L , and N_z denote the spatial resolution, temporal sequence length, and latent token count, respectively. By employing a parallel selective scan mechanism within the SSM blocks, the model circumvents the quadratic $O(L^2)$ bottleneck characteristic of standard Transformers, maintaining $O(L)$ scaling relative to the sequence length. Furthermore, our design effectively decouples the spatial resolution from the deep latent blocks by projecting the state onto a compact manifold where $N_z \ll N_p$ (e.g., $N_z \in \{64, 128\}$). This strategy significantly minimizes the memory footprint and latency during long-horizon rollouts, ensuring that the core spatiotemporal modeling remains computationally tractable even for high-fidelity simulations.

The robust performance of PGMNO is rooted in its adherence to classical numerical analysis principles. Theoretically, the accuracy of the framework is governed by the consistency of the embedded multistep scheme. By utilizing a second-order BDF-based structure ($k = 2$), the local truncation error of the temporal propagator is $O(\Delta t^2)$, which

provides a significantly higher fidelity for unsteady flows compared to first-order Euler-based mappings.

From a stability perspective, the framework inherits the **A-stability** of the BDF2 scheme, which is crucial for simulating stiff partial differential equations where high-frequency modes must be dissipated rather than amplified. Unlike standard recurrent architectures that may experience vanishing or exploding gradients during long-horizon rollouts, the structural coupling of the Mamba-based SSM with implicit integration ensures that the spectral radius of the transition operator is constrained. Specifically, the latent evolution is reformulated to satisfy:

$$\mathbf{h}_{i+k} = \Phi(\mathbf{h}_i, \dots, \mathbf{h}_{i+k-1}; \mathcal{N}), \quad (33)$$

where the mapping Φ is algebraically regularized to align with stable numerical trajectories. This synergy between state-space modeling and numerical stability criteria ensures that prediction errors are bounded within a controlled manifold, effectively suppressing the "causal collapse" analyzed in Fig. 6.

To contextualize the proposed framework within recent advancements, it is instructive to evaluate the performance of PGMNO alongside specialized architectures. The rapid evolution of neural operators has led to the emergence of specialized architectures such as the Transolver, which utilizes physics-aware attention to achieve linear scaling with respect to mesh points (Wu et al., 2024). While such models excel in spatial representational efficiency on complex geometries, PGMNO occupies a distinct niche by prioritizing temporal consistency through structural coupling. Unlike attention-based mechanisms that rely on learned long-range dependencies, PGMNO embeds an implicit BDF-based integration scheme that provides a formal stability guarantee, namely A-stability, which is often absent in purely data-driven transformers.

Recent latent PDE surrogates have also explored reduced-order modeling within compressed manifolds to accelerate simulations (Li et al., 2025). However, these frameworks typically employ standard recurrent or neural ODE-based mappings for temporal evolution, which may suffer from error accumulation in stiff or chaotic regimes. By contrast, our framework integrates the selective state-space modeling of Mamba with a multistep history window, effectively combining the memory capacity of SSMs with the rigorous error control of classical multistep methods. As discussed in recent state-space neural operator research (Koren & Lanthaler, 2025), the decoupling of spatial and temporal operators is key to scalability; PGMNO extends this concept by ensuring that the temporal evolution is not just learned, but algebraically constrained to follow stable numerical trajectories. Consequently, PGMNO provides a robust alternative to recent advancements by bridging the gap between high-performance sequence modeling and classical numerical stability criteria.

7. Conclusion

This paper presents the Physics-Guided Mamba Neural Operator (PGMNO), a novel operator learning framework that integrates linear multistep numerical integration with Structured State Space Models (SSMs) to address the longstanding challenge of long-term prediction in complex physical systems. By incorporating multistep temporal dependencies into the forward pass and employing an implicit Backward Differentiation Formula (BDF) scheme during training, PGMNO substantially improves predictive accuracy while ensuring numerical stability. The use of SSMs further enables efficient kernel operator approximation, enhancing both the scalability and representational capacity of the model. The effectiveness of the framework is demonstrated across a suite of canonical physical benchmarks. A key innovation of PGMNO lies in its explicit modeling of historical solution states through a multistep neural architecture. This design, combined with a BDF-informed training paradigm grounded in physical priors, reduces sample complexity and accelerates convergence. The training strategy enforces causality and temporal coherence, ensuring that accurate predictions at early time steps are reliably propagated throughout the entire solution trajectory.

Moreover, the model generalizes well across varying spatial discretizations, enabling resolution-invariant predictions for downstream tasks.

Looking forward, the PGMNO framework holds significant promise for tackling major challenges in computational science, especially in the long-term integration of partial differential equations (PDEs) for atmospheric and oceanic systems. Key applications, including climate modeling, weather forecasting, and ocean current prediction, require high-fidelity solutions to complex PDEs over extended time scales. PGMNO's inherent ability to capture long-range spatiotemporal dependencies and multiscale dynamics makes it a powerful candidate for creating the next generation of models in these vital fields. In essence, PGMNO offers a unified, robust, and scalable framework that seamlessly integrates classical numerical time-stepping with modern deep sequence modeling for operator learning. Its demonstrated accuracy, solid theoretical foundation, and architectural flexibility highlight its potential to significantly advance the field of scientific machine learning and enable data-driven discovery within complex physical systems.

CRedit authorship contribution statement

Yanan Guo: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Junqiang Song:** Project administration, Funding acquisition, Conceptualization; **Xiaoqun Cao:** Supervision, Project administration, Funding acquisition, Conceptualization; **Chuanfeng Zhao:** Project administration, Investigation; **Fukang Yin:** Project administration, Investigation; **Hongze Leng:** Supervision, Project administration, Investigation, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the [National Key Research and Development Program of China](#) [grant number 2021YFC3101500] and the [National Natural Science Foundation of China](#) [grant number 42276205].

Appendix A. Experimental Configurations and Hyperparameters

To facilitate the reproducibility of the results presented in [Section 5](#), we provide comprehensive specifications regarding physical parameters, neural network architectures, and training protocols. All PGMNO variants and baseline models were implemented using the PyTorch framework and executed on NVIDIA A100 GPUs to ensure consistent hardware performance.

A.1. Training protocols and optimizer settings

The optimization process for the PGMNO framework and all baseline architectures followed a standardized protocol to ensure an equitable performance comparison. We utilized the AdamW optimizer with an initial learning rate of 1×10^{-3} , which was adjusted during training via a Cosine Annealing scheduler. The specific hyperparameters and dataset partitions are summarized in [Table A.4](#).

A.2. Architectural details and baseline alignment

The PGMNO framework consists of a lifting operator, multiple latent SSM blocks, and a decoding projection layer. To ensure a rigorous benchmarking environment, baseline models such as FNO and DeepONet were scaled to ensure that their total trainable parameters remained within a $\pm 5\%$ range of the PGMNO configuration. This alignment ensures that

Table A.4

Standardized training hyperparameters and dataset specifications across all benchmarks.

Hyperparameter	Burgers	R-B	Kolmogorov	S-W
Training Samples	1000	500	800	200
Validation/Test Samples	200/300	100/150	100/100	50/75
Forecast Horizon (T)	100	50	30	50
Total Training Epochs	500	500	500	500
Batch Size	32	32	16	16
Weight Decay	1×10^{-4}	1×10^{-4}	1×10^{-4}	1×10^{-4}
Learning Rate Scheduler	Cosine	Cosine	Cosine	Cosine
Min Learning Rate	1×10^{-6}	1×10^{-6}	1×10^{-6}	1×10^{-6}

Table A.5

Architectural specifications and structural settings for the latent PGMNO blocks.

Setting	Burgers	R-B	Kolmogorov	S-W
Number of Latent Layers	4	6	8	6
Latent Dimension (N_z)	64	128	128	96
A-matrix Initialization	HiPPO-LegS	HiPPO-LegS	HiPPO-LegS	HiPPO-LegS
Selective Scan Implementation	Parallel	Parallel	Parallel	Parallel
Convolutional Kernel Size	4	4	4	4
Expansion Factor	2	2	2	2
Normalization Layer	LayerNorm	LayerNorm	LayerNorm	LayerNorm

the observed performance gains are attributable to the methodological integration of numerical schemes rather than discrepancies in model capacity. The structural configurations for the PGMNO blocks are detailed in [Table A.5](#).

A.3. Implementation and computational stability

To stabilize the training of SSM blocks on stiff dynamics, we initialized the A-matrices using the HiPPO-LegS scheme, which facilitates the modeling of long-range temporal dependencies. Furthermore, we replaced standard BatchNorm with LayerNorm to eliminate the overhead associated with frequent spatial reshaping in latent space. Statistical evaluations across all tasks are reported as the mean and standard deviation derived from three independent runs with fixed random seeds: 42, 123, and 456.

References

- Abdulsitar, A. S. (2025). Comparative analysis of FTCS, richardson, and dufort-frankel numerical methods for temperature distribution in a one-dimensional rod. *Babylonian Journal of Mechanical Engineering*, 2025, 98–103.
- Adamopoulos, I., Valamontes, A., Karantonis, J. T., Tropaitis, P. P., & Syrou, N. (2025). Enhancing economic growth time series for UAE forecasting with deep learning: A seq2seq and attention-driven LSTM approach. *EDRAAK*, 2025, 62–75.
- Ames, W. F. (2014). Numerical methods for partial differential equations. Academic press.
- Anandkumar, A., Azizzadenesheli, K., Bhattacharya, K., Kovachki, N., Li, Z., Liu, B., & Stuart, A. (2019). Neural operator: Graph kernel network for partial differential equations. In *Icml 2020 workshop on integration of deep neural models and differential equations*. <https://openreview.net/forum?id=fg2ZFmXFO3>.
- Asaithambi, A. (2010). Numerical solution of the burgers' equation by automatic differentiation. *Applied Mathematics and Computation*, 216(9), 2700–2708.
- Bar-Sinai, Y., Hoyer, S., Hickey, J., & Brenner, M. P. (2019). Learning data-driven discretizations for partial differential equations. *Proceedings of the National Academy of Sciences*, 116(31), 15344–15349.
- Chung, S. W., Choi, Y., Miller, C., Springer, H. K., & Sullivan, K. T. (2026). Latent space element method. *arXiv preprint arXiv:2601.01741*.
- Dal Santo, N., Deparis, S., & Pegolotti, L. (2020). Data driven approximation of parametrized PDEs by reduced basis and neural networks. *Journal of Computational Physics*, 416, 109550.
- Evans, L. C. (2022). Partial differential equations (vol. 19). American mathematical society.
- Fefferman, C. L., Robinson, J. C., Diez, J. L. R., & Rodrigo, J. L. (2018). Partial differential equations in fluid mechanics (vol. 452). Cambridge University Press.
- Galevsky, J., Scott, R. K., & Polvani, L. M. (2004). An initial-value problem for testing numerical models of the global shallow-water equations. *Tellus A: Dynamic Meteorology and Oceanography*, 56(5), 429–440.
- Guo, Y., Cao, X., Song, J., Leng, H., & Peng, K. (2023). An efficient framework for solving forward and inverse problems of nonlinear partial differential equations via enhanced physics-informed neural network based on adaptive learning. *Physics of Fluids*, 35(10).

- Hakim, B. A. (2024). Using of computational fluid dynamics (CFD) and machine learning (ML) for predictive modelling in fluid dynamic. *Khwarizmia*, 2024, 85–94.
- Huang, S., Feng, W., Tang, C., He, Z., Yu, C., & Lv, J. (2025). Partial differential equations meet deep neural networks: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, .
- Johnson, C. (2009). Numerical solution of partial differential equations by the finite element method. Courier Corporation.
- Koren, N., & Lanthaler, S. (2025). Merging memory and space: A state space neural operator. arXiv preprint arXiv:2507.23428.
- Kovachki, N., Lanthaler, S., & Mishra, S. (2021). On universal approximation and error bounds for fourier neural operators. *Journal of Machine Learning Research*, 22(290), 1–76.
- Kovachki, N., Li, Z., Liu, B., Azizzadenesheli, K., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2023). Neural operator: Learning maps between function spaces with applications to pdes. *Journal of Machine Learning Research*, 24(89), 1–97.
- Kumar, V., Goswami, S., Kontolati, K., Shields, M. D., & Karniadakis, G. E. (2025). Synergistic learning with multi-task deepONet for efficient PDE problem solving. *Neural Networks*, 184, 107113.
- Li, D., Tan, S., Zhang, Y., Jin, M., Pan, S., Okumura, M., & Jiang, R. (2024). Dyg-mamba: Continuous state space modeling on dynamic graphs. arXiv preprint arXiv:2408.06966.
- Li, Z., Huang, D. Z., Liu, B., & Anandkumar, A. (2023a). Fourier neural operator with learned deformations for pdes on general geometries. *Journal of Machine Learning Research*, 24(388), 1–26.
- Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Stuart, A., Bhattacharya, K., & Anandkumar, A. (2020). Multipole graph neural operator for parametric partial differential equations. *Advances in Neural Information Processing Systems*, 33, 6755–6766.
- Li, Z., Patil, S., Ogoke, F., Shu, D., Zhen, W., Schneier, M., Buchanan, J. R., Jr, & Farimani, A. B. (2025). Latent neural PDE solver: A reduced-order modeling framework for partial differential equations. *Journal of Computational Physics*, 524, 113705.
- Li, Z., Shu, D., & Barati Farimani, A. (2023b). Scalable transformer for pde surrogate modeling. *Advances in Neural Information Processing Systems*, 36, 28010–28039.
- Luo, K., Zhao, J., Wang, Y., Li, J., Wen, J., Liang, J., Soekmadji, H., & Liao, S. (2025). Physics-informed neural networks for PDE problems: a comprehensive review. *Artificial Intelligence Review*, 58(10), 1–43.
- Luo, Y., Peng, J., & Ma, J. (2020). When causal inference meets deep learning. *Nature Machine Intelligence*, 2(8), 426–427.
- Muhammad, A., & Khan, T. A. (2024). Numerical solution of burgers'-type equations using modified variational iteration algorithm-i. *Babylonian Journal of Mathematics*, 2024, 34–43.
- Olver, P. J. et al. (2014). Introduction to partial differential equations (vol. 1). Springer.
- Overview of neural networks. *Babylonian Journal of Machine Learning*, 2023,(2023) 42–45. <https://journals.mesopotamian.press/index.php/BJML/article/view/264>. <https://doi.org/10.58496/BJML/2023/008>
- Rehman, W., De Azevedo, F. S., & Ullah, S. (2025). Chemical reaction and activation energy impacts on unsteady radiative flow of sisko fluid. *Babylonian Journal of Mathematics*, 2025, 1–11.
- Sadia, R. T., Ahamed, M. A., & Cheng, Q. (2025). Causalged: blending causality and diffusion for spatial gene expression generation. In *Proceedings of the 31st ACM SIGKDD conference on knowledge discovery and data mining* v. 2 (pp. 2466–2477).
- Song, J.-J., Li, P.-X., Chen, L., Li, C.-H., Li, B.-W., & Huang, L.-Y. (2023). A review on rayleigh-bénard convection influenced by the complicating factors. *International Communications in Heat and Mass Transfer*, 144, 106784.
- Su, L., Zuo, X., Li, R., Wang, X., Zhao, H., & Huang, B. (2025). A systematic review for transformer-based long-term series forecasting. *Artificial Intelligence Review*, 58(3), 80.
- Thomas, J. W. (2013). Numerical partial differential equations: finite difference methods (vol. 22). Springer Science & Business Media.
- Wang, T., & Wang, C. (2024). Latent neural operator for solving forward and inverse pde problems. *Advances in Neural Information Processing Systems*, 37, 33085–33107.
- Wu, H., Luo, H., Wang, H., Wang, J., & Long, M. (2024). Transolver: A fast transformer solver for pdes on general geometries. arXiv preprint arXiv:2402.02366.
- Zha, W., Zhang, W., Li, D., Xing, Y., He, L., & Tan, J. (2022). Convolution-based model-solving method for three-dimensional, unsteady, partial differential equations. *Neural Computation*, 34(2), 518–540.
- Zhongkai, H., Yao, J., Su, C., Su, H., Wang, Z., Lu, F., Xia, Z., Zhang, Y., Liu, S., Lu, L. et al. (2024). Pinnacle: A comprehensive benchmark of physics-informed neural networks for solving pdes. *Advances in Neural Information Processing Systems*, 37, 76721–76774.